

Sistem Operasi (INF-201)

Pertemuan 4: Penjadwalan CPU (CPU Scheduling)

Tim Dosen Sistem Operasi

Program Studi Informatika – Universitas Bengkulu

Semester Ganjil 2026/2027

Tanggal Kompilasi: 8 September 2026



Capaian Pembelajaran (Sub-CPMK 4)

- Mahasiswa mampu menganalisis siklus CPU-I/O Burst dan kriteria performa penjadwalan.
- Mahasiswa mampu mengkonstruksi **Gantt Chart** dan menghitung metrik *Waiting Time (W)*, *Turnaround Time (TAT)*, dan *Response Time*.
- Mahasiswa mampu membandingkan algoritma FCFS, SJF (Non-preemptive), SRTF (Preemptive), dan Round Robin.
- Mahasiswa memahami mekanisme adaptif **Multilevel Feedback Queue (MLFQ)** untuk mengoptimalkan *TAT* dan *Response Time* secara simultan.

Siklus CPU-I/O Burst & Metrik Evaluasi Penjadwalan

Siklus Hidup Proses (CPU-I/O Burst Cycle)

Eksekusi proses terdiri dari perulangan fase komputasi (*CPU Burst*) dan fase tunggu input/output (*I/O Burst*).

Metrik Evaluasi Sistem Operasi:

- **CPU Utilization:** % Waktu CPU sibuk (max 100%).
- **Throughput:** Jumlah proses selesai per unit waktu.
- **Turnaround Time (TAT):**

$$TAT = T_{\text{selesai}} - T_{\text{datang}}$$

Metrik Pengalaman Pengguna:

- **Waiting Time (W):** Total waktu di Ready Queue:

$$W = TAT - \text{Burst Time}$$

- **Response Time:** Waktu dari submit hingga respon pertama CPU muncul.

Non-Preemptive vs Preemptive Scheduling

Non-Preemptive Scheduling

Sekali CPU dialokasikan ke suatu proses, proses tersebut memegang kendali CPU hingga selesai (*Terminated*) atau sukarela melepas CPU untuk I/O (*Waiting*).

- Lebih sederhana, zero race condition di scheduler.
- Contoh: **FCFS, SJF Murni**.

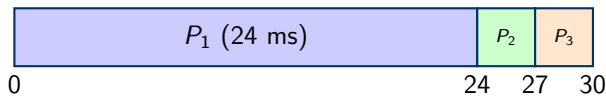
Preemptive Scheduling

CPU dapat diambil paksa (*preempt*) oleh OS di tengah eksekusi proses ketika ada interupsi timer habis atau proses berprioritas lebih tinggi tiba.

- Responsif untuk sistem interaktif & multitasking.
- Contoh: **Round Robin, SRTF, MLFQ**.

Algoritma 1: FCFS & Fenomena Convoy Effect

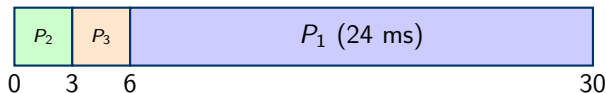
Studi Kasus: P_1 ($Burst = 24$), P_2 ($Burst = 3$), P_3 ($Burst = 3$) tiba di $t = 0$.



- $W(P_1) = 0$, $W(P_2) = 24$, $W(P_3) = 27 \implies$ **Rata-rata Waiting Time = 17.0 ms.**
- **Convoy Effect:** Proses CPU-bound panjang (P_1) memblokir seluruh proses I/O-bound pendek (P_2, P_3), menurunkan throughput dan membuat respons sistem lambat.

Algoritma 2: SJF (Shortest Job First) Non-Preemptive

Studi Kasus yang Sama dengan Urutan SJF: $P_2(3) \rightarrow P_3(3) \rightarrow P_1(24)$.

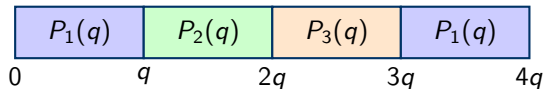


- $W(P_2) = 0$, $W(P_3) = 3$, $W(P_1) = 6 \implies$ **Rata-rata Waiting Time = 3.0 ms** (Turun 82%!).
- **Optimalitas SJF:** Terbukti secara matematis menghasilkan rata-rata waiting time terkecil.
- **Tantangan Praktis:** OS tidak bisa mengetahui panjang burst berikutnya secara pasti.

Algoritma 3: Round Robin (RR) & Time Quantum (q)

Mekanisme Round Robin

Setiap proses di Ready Queue diberikan jatah waktu eksekusi CPU tetap bernama **Time Quantum** (q). Jika proses belum selesai saat q habis, terjadi interupsi timer $\rightarrow$ proses di-preempt dan dipindahkan ke ujung belakang antrean.



Trade-off Pemilihan Nilai q :

- q terlalu besar ($q \rightarrow \infty$): RR berubah menjadi FCFS (Convoy effect).
- q terlalu kecil ($q \rightarrow 0$): CPU menghabiskan sebagian besar waktu untuk *Context Switch Overhead*.
- **Rule of Thumb:** 80% CPU burst harus lebih pendek daripada nilai q (umumnya 10 – 100 ms).

Priority Scheduling & Masalah Starvation

Priority Scheduling

Setiap proses diasosiasikan dengan nilai prioritas integer P . CPU dialokasikan ke proses dengan prioritas tertinggi.

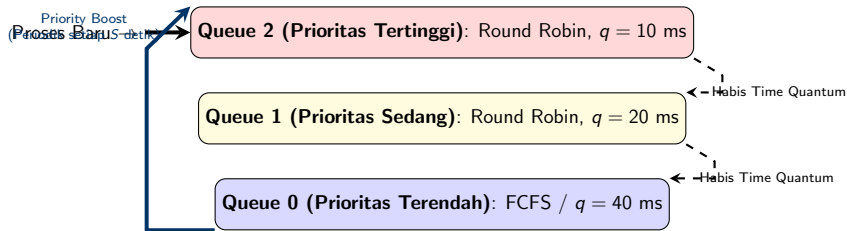
- Preemptive atau Non-preemptive.
- Masalah Utama: **Indefinite Blocking / Starvation** (proses prioritas rendah tidak pernah jalan jika proses prioritas tinggi terus berdatangan).

Solusi: Priority Aging

Secara bertahap meningkatkan prioritas proses seiring berjalannya waktu tunggu di Ready Queue.

- Contoh: Proses dengan prioritas 100 dinaikkan 1 tingkat setiap 1 menit menunggu.
- Dijamin dalam waktu tertentu proses akan menjadi prioritas utama (#1) dan dieksekusi.

Arsitektur MLFQ (Multilevel Feedback Queue)



MLFQ belajar secara otomatis: Proses interaktif (I/O-bound) tetap di Q2/Q1 karena sering melepas CPU sebelum q habis; proses komputasi panjang (CPU-bound) otomatis turun ke Q0.

5 Aturan Emas MLFQ (OSTEP Ch. 8)

Aturan Lengkap Multi-Level Feedback Queue Scheduler

- ① **Rule 1:** Jika $\text{Priority}(A) > \text{Priority}(B)$, A berjalan (B tidak).
- ② **Rule 2:** Jika $\text{Priority}(A) = \text{Priority}(B)$, A & B berjalan dalam Round Robin menggunakan time-slice antrian tersebut.
- ③ **Rule 3:** Saat proses baru memasuki sistem, diletakkan di antrian prioritas tertinggi (asumsi awal adalah proses pendek/interaktif).
- ④ **Rule 4 (Accounting):** Setelah proses menghabiskan jatah waktunya di satu tingkat (terlepas berapa kali ia melepaskan CPU), prioritasnya diturunkan 1 tingkat.
- ⑤ **Rule 5 (Priority Boost):** Setelah periode waktu S , pindahkan **semua** proses ke antrian paling atas (mencegah *Starvation* dan mengakomodasi proses yang berubah dari CPU-bound menjadi I/O-bound).

Tantangan Utama Sistem Multi-Core

Pada CPU modern dengan banyak inti (N -Core), setiap core memiliki **Hardware Cache (L1/L2)** masing-masing. Memindahkan proses sembarangan antar-core akan menghancurkan performa cache!

1. SQMS (Single-Queue):

- Satu antrean global untuk semua core CPU.
- Masalah: Butuh penguncian (*synchronization lock*) yang berat dan merusak **Cache Affinity**.

2. MQMS (Multi-Queue):

- Setiap core CPU memiliki antrean penjadwalan lokal sendiri.
- Menjaga **Cache Affinity** secara optimal.
- Memerlukan mekanisme **Work Stealing** untuk menyeimbangkan beban (*Load Balancing*).

Penjadwal Dunia Nyata: Linux Completely Fair Scheduler (CFS)

Filosofi Desain Linux CFS

Alih-alih menggunakan fixed time-slice klasik, Linux CFS mensimulasikan prosesor ideal di mana setiap proses menerima porsi CPU yang persis seimbang (*Proportional Share*).

Konsep Virtual Runtime (vruntime):

- Setiap proses memiliki variabel `vruntime` yang mengukur berapa lama proses telah menggunakan CPU fisik.
- Proses dengan `vruntime` terendah selalu dipilih untuk dieksekusi berikutnya.

Struktur Data Red-Black Tree:

- Ready Queue diimplementasikan menggunakan **Red-Black Tree** (Self-balancing Binary Search Tree).
- Proses dengan `vruntime` terkecil berada di simpul paling kiri ($O(\log N)$ insert, $O(1)$ pick min).
- Prioritas dikontrol via nilai **nice value** (-20 hingga $+19$).

Perbandingan Komprehensif Algoritma Penjadwalan

Algoritma	Tipe	Keunggulan Utama	Kelemahan Utama
FCFS	Non-preemptive	Sangat sederhana, overhead nol	Convoy effect, average waiting time buruk
SJF / SRTF	Non/Preemptive	Optimal untuk minimisasi W	Sulit prediksi burst, rawan starvation
Round Robin	Preemptive	Responsif, adil untuk timesharing	TAT rata-rata lebih tinggi jika burst serupa
MLFQ	Preemptive	Adaptif tanpa butuh info durasi masa depan	Perlu tuning parameter (q , S , jumlah antrean)

Tugas Membaca Mingguan (Reading Assignment)

Persiapan Pertemuan 5: Sinkronisasi Proses & Multi-threading

Sebelum mengikuti kuliah minggu depan, mahasiswa wajib membaca:

- **[OSTEP]** Chapter 26: *Concurrency: An Introduction* (hal. 291–304)
- **[OSTEP]** Chapter 28: *Locks* (hal. 319–336)
- **[OSTEP]** Chapter 31: *Semaphores* (hal. 363–378)
- **[Think OS]** Chapter 7–9: *Threads, Mutex, Semaphores* (hal. 55–82)
- **[xv6 RISC-V]** Chapter 6: *Locking* (hal. 63–72)

Pertanyaan Pemicu Diskusi Mandiri

Mengapa instruksi biasa seperti `counter++` di C dapat menimbulkan *Data Race* ketika dijalankan oleh dua thread berbeda secara konkuren?